



Report

Design and Implementation of a GCD Computation Unit

Chenga Reddy Bhargavi



Report

Design and Implementation of a GCD Computation Unit

1. VLSI Architecture and Top Module

System Overview

The GCD Computation Unit is structured as a classic two-block VLSI architecture: a Datapath block that performs arithmetic and stores state, and a Controller block (FSM) that sequences operations by asserting control signals to the datapath. The two blocks communicate through a well-defined set of status signals (from datapath to controller) and control signals (from controller to datapath). The top module `gcd_top` instantiates both blocks and connects them.

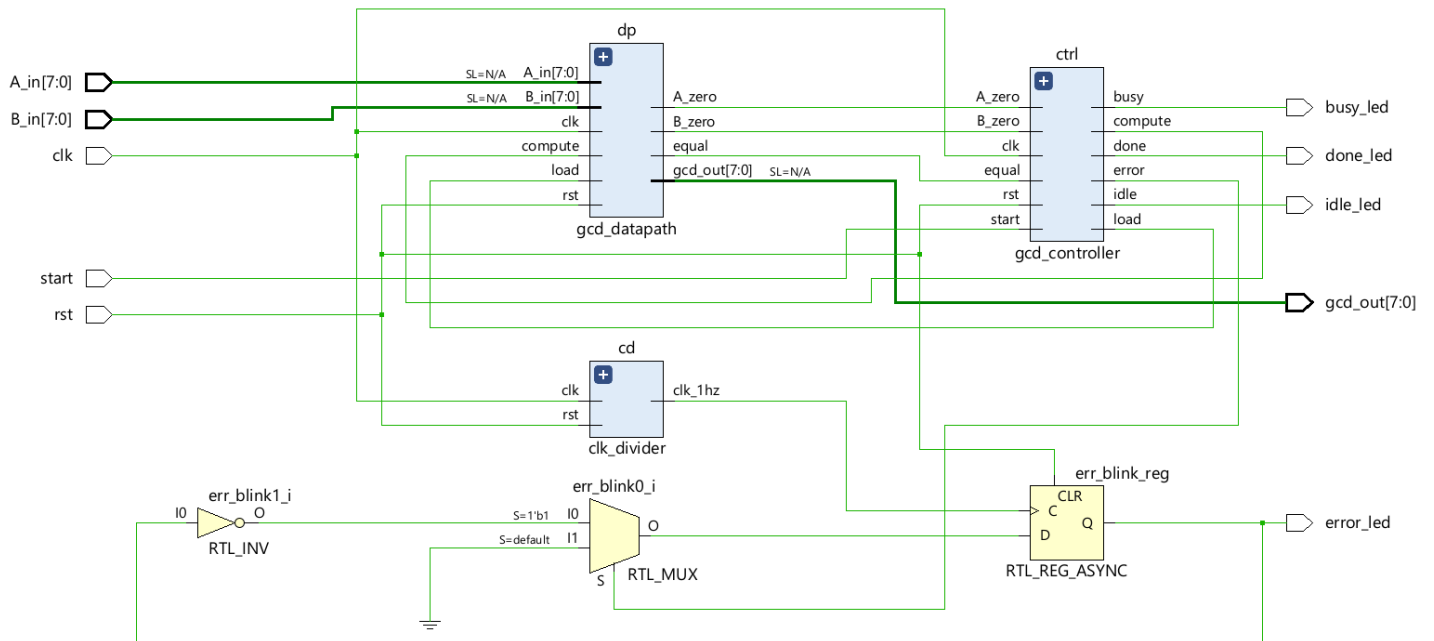
Top Module Port Diagram

Signal	Width	Direction	Description
<code>clk</code>	1-bit	Input	System clock, positive-edge triggered (100 MHz)
<code>rst</code>	1-bit	Input	Synchronous active-HIGH reset; returns FSM to IDLE
<code>start</code>	1-bit	Input	Latches <code>A_in/B_in</code> and triggers computation
<code>A_in</code>	8-bit	Input	First operand (0–255), sampled when <code>start</code> is asserted
<code>B_in</code>	8-bit	Input	Second operand (0–255), sampled when <code>start</code> is asserted
<code>gcd_out</code>	8-bit	Output	Computed GCD result; stable when <code>done</code> is HIGH
<code>done</code>	1-bit	Output	Pulsed HIGH for one clock cycle when result is ready
<code>busy</code>	1-bit	Output	HIGH from <code>LOAD</code> through final <code>COMPARE</code> state
<code>idle</code>	1-bit	Output	HIGH when FSM is in <code>IDLE</code> state
<code>error</code>	1-bit	Output	Toggled when both <code>A_in</code> and <code>B_in</code> are zero

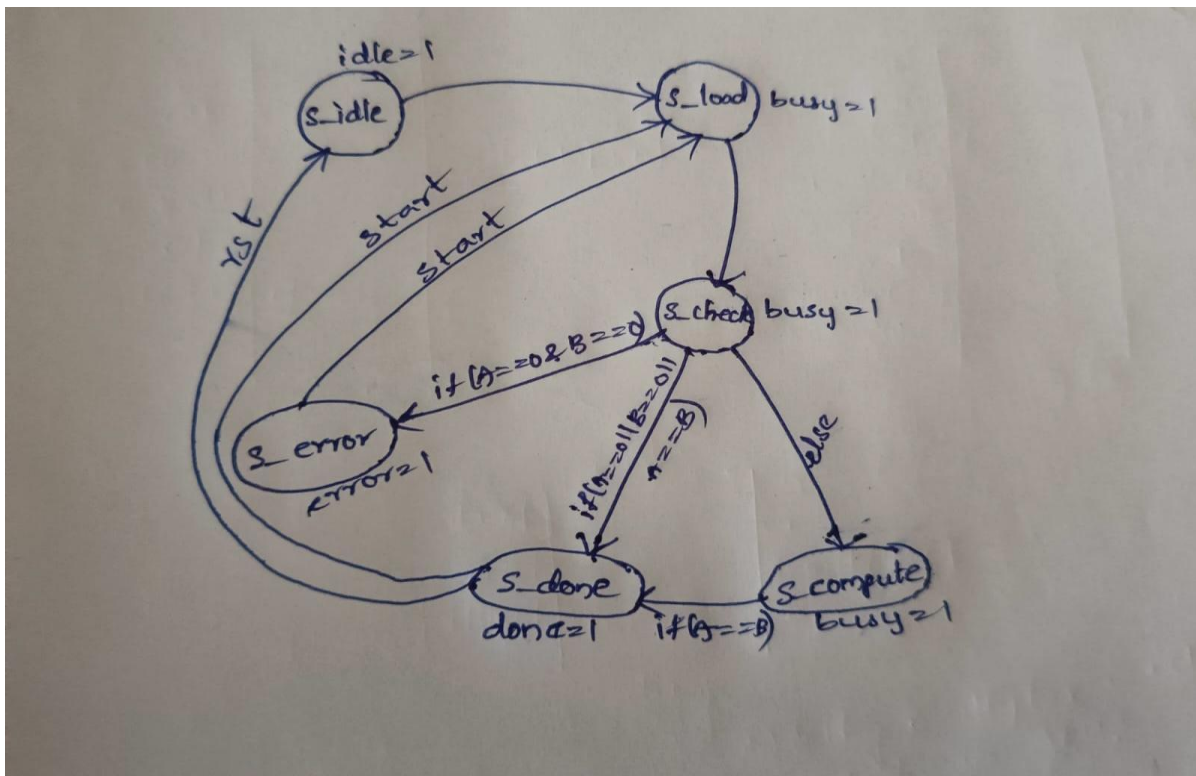
Architecture Block Diagram

The diagram below shows the top-level structure of `gcd_top` with the two sub-modules and all interconnecting signals.

The `gcd_top` module does not perform any computation itself — it just connects the `datapath` and `controller` modules together. All external inputs like `clk`, `rst`, `start`, `A_in`, and `B_in` are passed down to the two sub-modules as needed. The controller acts as the brain, sending control signals like `load`, `sub_a`, and `sub_b` to tell the datapath what operation to perform each clock cycle. The datapath sends back status signals like `A_gt_B` and `A_eq_B` so the controller can decide the next state. Finally, outputs like `gcd_out`, `done`, and `busy` are directly wired from the sub-modules to the top-level ports.



STATE DIAGRAM:



Datapath–Control Unit Interfacing

The controller asserts one of four mutually exclusive control signals each clock cycle. The datapath responds combinationally (comparator outputs) and registers the result on the next rising edge. The handshake is summarised below:

Signal	Direction	Role
load	CTR → DP	Latch A_in / B_in into registers A and B
sub_a	CTR → DP	Compute $A \leftarrow A - B$ on next rising edge
sub_b	CTR → DP	Compute $B \leftarrow B - A$ on next rising edge
capture	CTR → DP	Latch result into gcd_out register (asserted in COMPARE when terminal condition met)
A_gt_B	DP → CTR	Combinational: $A > B$
A_eq_B	DP → CTR	Combinational: $A = B$
A_zero	DP → CTR	Combinational: $A = 0$
B_zero	DP → CTR	Combinational: $B = 0$
both_zero	DP → CTR	Combinational: $A_{in} = 0 \text{ AND } B_{in} = 0$ (error condition)

Datapath Operation by State

FSM State	Datapath Action
IDLE	No register updates. Comparator evaluates A_in/B_in continuously (both_zero).
LOAD	load=1: $A \leftarrow A_{in}$, $B \leftarrow B_{in}$ on rising edge.
COMPARE	Comparator evaluates A, B → drives A_gt_B, A_eq_B, A_zero, B_zero. If terminal condition met, capture=1: gcd_out ← B (if A=0) or A (otherwise).
SUB_A	sub_a=1: $A \leftarrow A - B$ on rising edge.
SUB_B	sub_b=1: $B \leftarrow B - A$ on rising edge.
DONE	No datapath update. gcd_out is stable and valid.
ERROR	No datapath update. gcd_out retains previous value.

Control Unit FSM

State Encoding

The controller uses a 3-bit binary state register with seven states. The FSM is a Moore machine — outputs depend only on the current state, except for the capture signal which is conditionally asserted in COMPARE based on the terminal condition.

State	Encoding	Description
IDLE	3'b000	Waiting for start. Asserts idle=1.
LOAD	3'b001	Latches operands into registers A and B.
COMPARE	3'b010	Evaluates termination conditions; asserts capture if done.
SUB_A	3'b011	Performs $A \leftarrow A - B$.
SUB_B	3'b100	Performs $B \leftarrow B - A$.
DONE	3'b101	Pulses done=1 for one clock cycle.
ERROR	3'b110	Toggles error flag when both inputs are zero.

Full State Transition Table

State (Encoding)	Moore Outputs	Next-State Transitions
IDLE (000)	idle=1, all others 0	start && both_zero → ERROR start → LOAD else → IDLE
LOAD (001)	load=1, busy=1	→ COMPARE (unconditional)
COMPARE (010)	busy=1; capture=1 if done cond	A_eq_B A_zero B_zero → DONE A_gt_B → SUB_A else → SUB_B
SUB_A (011)	sub_a=1, busy=1	→ COMPARE (unconditional)
SUB_B (100)	sub_b=1, busy=1	→ COMPARE (unconditional)
DONE (101)	done=1 (1-cycle pulse)	→ IDLE (unconditional)
ERROR (110)	error=1 (toggle)	→ IDLE (unconditional)

4. Resource Utilisation

Vivado Implementation Summary

The design was synthesised and implemented in Vivado targeting the Xilinx XC7S50 Spartan-7 FPGA on the Boolean Board. The utilisation report (utilization_1) from the post-implementation run is summarised below.

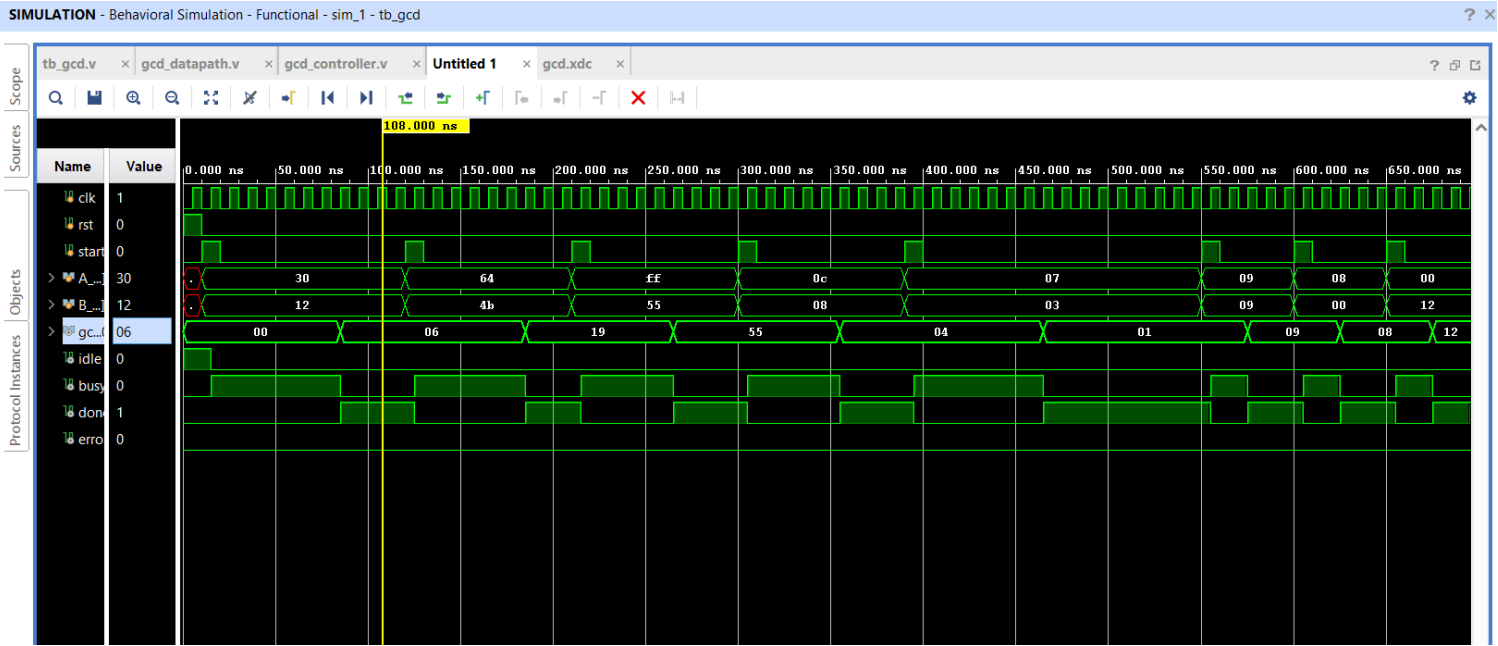
Resource	Used	Available	Utilisation
Slice LUTs	38	32,600	0.12%
Slice Registers	32	65,200	0.05%
Slices	14	8,150	0.17%
LUT as Logic	38	32,600	0.12%
Bonded IOBs	31	210	14.76%
BUFGCTRL	1	32	3.13%

Simulation Results Analysis:

Behavioral simulation was performed in Vivado to verify the functionality of the GCD Computation Unit. Multiple test vectors were applied through the testbench, and the generated outputs were compared against the expected Greatest Common Divisor values.

Test Case	A Input (Dec)	B Input (Dec)	Expected GCD	Simulated GCD	Result
1	48	18	6	6	Pass
2	100	75	25	25	Pass
3	255	85	85	85	Pass
4	12	8	4	4	Pass
5	7	3	1	1	Pass
6	9	9	9	9	Pass
7	8	0	8	8	Pass
8	0	18	18	18	Pass

The simulation waveform confirms correct operation of the controller and datapath. After the start signal is asserted, the FSM loads the input operands and repeatedly performs subtraction operations until a termination condition is reached. The busy signal remains asserted while computation is in progress and is deasserted once the result becomes available. The done signal generates a pulse indicating successful completion of the operation. For all test cases, the computed output matched the mathematically expected GCD value, demonstrating correct implementation of the Euclidean subtraction algorithm and proper interaction between the datapath and controller modules.



5. XDC Constraints File Description

Overview

The Xilinx Design Constraints (XDC) file maps all top-module ports to physical FPGA package pins on the Boolean Board and sets the required I/O standard. All I/O pins are configured as LVCMOS33 (3.3 V logic), consistent with the Boolean Board's bank voltage.

Pin Assignment Table

Port	FPGA Pin(s)	Direction	Board Resource
clk	F14	Input	100 MHz on-board oscillator
rst	J2	Input	On-board push-button (active HIGH)
start	J5	Input	On-board push-button
A_in[7:0]	P2–V2	Input	Slide switches SW7–SW0
B_in[7:0]	K1–P1	Input	Slide switches SW15–SW8
gcd_out[7:0]	E5–G1	Output	On-board LEDs LD7–LD0
idle	A4	Output	Status LED
busy	B4	Output	Status LED
done	A3	Output	Status LED
error	B3	Output	Status LED

IOSTANDARD

All pins use LVCMOS33 (Low-Voltage CMOS, 3.3 V). This matches the Boolean Board's 3.3 V I/O bank supply voltage and is the required standard for compatibility with the on-board switches, buttons, and LEDs.